The PyCore CALL Finite-State Machine

Shared argument binding for CPython 3.14 bytecode on a research CPU

PyCore systems notes
`docs/paper/systems/call_fsm.tex`

August 2026

Abstract

PyCore executes a CPython 3.14 bytecode subset in hardware. Function invocation is one of the most semantics-dense paths in that subset: free functions, bound methods, keyword calls, starred calls, defaults, positional-only parameters, `*args`, and `**kwargs` must all agree with CPython before higher-level components can rely on ordinary Python call shapes. This note documents the `S_CALL` finite-state machine and its *shared binder*—the hardware path that maps caller stack arguments onto callee locals. We describe phase structure, call modes, stack layouts, binder micro-sequences, worked examples, design tradeoffs, and remaining limits. Primary RTL sources are `pycore/rtl/pycore_call_fsm.svh` and `pycore/rtl/pycore_core.sv`.

Contents

1 Motivation and scope

PyCore's ISA is not a conventional RISC; it is a filtered CPython bytecode. That choice makes *call* a first-class microarchitectural concern rather than a compiler ABI detail. CPython 3.14 emits three related opcodes for ordinary calls:

Opcode	Hex	Role
`CALL`	52	Positional arguments only
`CALL_KW`	55	Mixed positional + keyword (names tuple at TOS)
`CALL_FUNCTION_EX`	04	`*args` / `**kwargs` expand

A fourth opcode, `DICT_MERGE`, often precedes `CALL_FUNCTION_EX` when the caller writes `f(*a, **d)`. PyCore implements merge separately; this document focuses on the CALL FSM that consumes the resulting stack shape.

Why a dedicated FSM. Argument binding is multi-cycle: it reads code-object metadata from dmem, probes name tuples and default containers, may allocate an `*args` tuple or a `**kwargs` dict, then pushes a hardware frame. Folding that work into a single "execute" beat is neither faithful to CPython nor practical under PyCore's memory ports. The FSM therefore sits as core state `S_CALL`, with an internal phase register and a nested binder sub-state machine.

Scope of this note. We cover user `CODE_OBJECT` callees (the interim model where a function *is* a code-object handle), method-form calls, bound-method unwrap, type instantiation entry, and the on-core builtin fast path at a high level. We do not re-derive excore grow handlers or the full object model; those are sibling systems notes.

2 Architectural placement

Figure 1 places CALL in the core control loop. Decode recognizes `CALL` / `CALL_KW` / `CALL_FUNCTION_EX` and enters `S_CALL`. On success the FSM ends in `CALL_PHASE_DONE`, which redirects fetch to the callee entry and leaves the value stack ready for the callee body. On failure it raises a trap—most often `PY_TRAP_CALL_FILTER` (code 6), PyCore's fatal stand-in for CPython `TypeError` on bad call shapes.

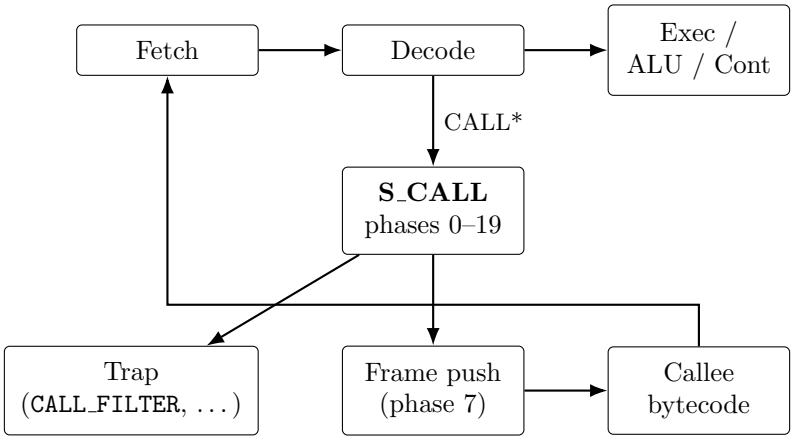

Figure 1: CALL in the PyCore control hierarchy. `S_CALL` owns argument normalization and binding; frame push is a distinct phase that consumes the binder's effective argc.

3 Call modes

`call_mode_r` records how the caller presented arguments:

Mode	Value	Meaning
`CALL_MODE_POS`	0	Plain `CALL`; oparg = positional count
`CALL_MODE_KW`	1	`CALL_KW`; TOS is names `TUPLE`
`CALL_MODE_EX`	2	`CALL_FUNCTION_EX` expand in progress
`CALL_MODE_EX_KW`	3	EX path with a kwargs `MUT_DICT`

`CALL_FUNCTION_EX` always begins in mode `EX`. After the kwargs sentinel/dict and args list/tuple are classified, the FSM either joins the positional path (`POS`) or the keyword binder (`EX_KW`). That join is intentional: once starred arguments are expanded onto the stack, binding is the same problem as `CALL`/`CALL_KW`.

4 Phase map

Figure 2 is the top-level phase automaton. Phases 0–7 are the common `CODE_OBJECT` path. Phases 8–13 handle non-code callables. Phases 16–19 are entry preludes for keyword and starred forms.

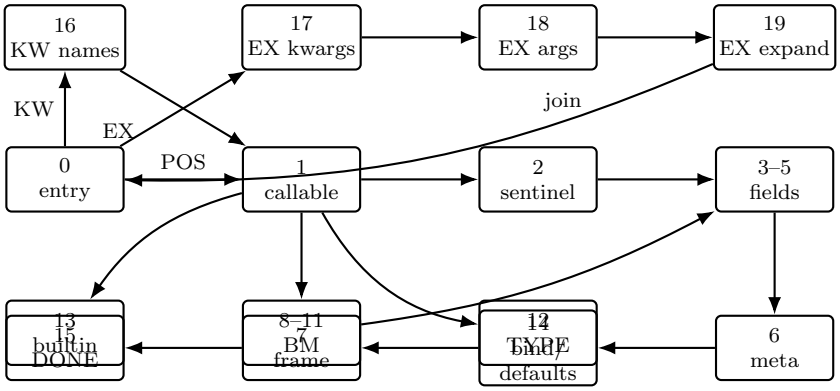

Figure 2: Top-level `call_phase_r` map (simplified). Object/builtin/type arms rejoin the code-object field reads or complete in-place without a Python frame.

4.1 Phase responsibilities

`0` Mode dispatch: compute stack bases for POS, or divert to KW/EX preludes.

`1` Classify callable tag: `CODE_OBJECT`, `OBJECT` (bound method / type), else `CALL_FILTER`.

`2` Free vs. method-form via the NULL sentinel; set effective argc.

`3--5` Read `entry_slot`, `co_consts`, `co_names`.

`6` Latch metadata (argcount, nlocals, kwonly, varargs, varkw, posonly); start `co_defaults` read; choose binder vs. positional-defaults arm.

`7` Frame push; ranged UNINIT clear of unfilled locals [`call_argcount`, `nlocals`).

`8--11` Bound-method unwrap to underlying code + self.

`12` Type instantiation / `__init__` lookup.

13 OBK_BUILTIN fast path (max/len/range, ...).

14 Positional defaults fill *or* shared keyword binder.

16--19
CALL_KW / CALL_FUNCTION_EX stack normalization.

5 Stack layouts

CPython 3.14's calling convention is preserved on the hardware value stack (register file slots addressed from TOS).

5.1 Free function

```
// callable @ RF[tos - oparg - 2]
// null @ RF[tos - oparg - 1] // CONTROL+NULL
// args @ RF[tos - oparg .. tos - 1]
```

5.2 Method form

Produced by LOAD_ATTR with the method flag set:

```
// callable @ RF[tos - oparg - 2]
// self @ RF[tos - oparg - 1] // non-NULL
// args @ RF[tos - oparg .. tos - 1]
```

5.3 CALL_KW

TOS is a names TUPLE; below it lie keyword values (names order), then positionals, then the free/method prefix. Phase 16 pops the names tuple and sets `n_pos = oparg - n_kwargs`.

5.4 CALL_FUNCTION_EX

TOS is either NULL (no kwargs) or a kwargs dict; below it sits the args list/tuple. Phases 17–19 classify, expand elements onto the stack, then rejoin phase 0 in POS or EX_KW mode.

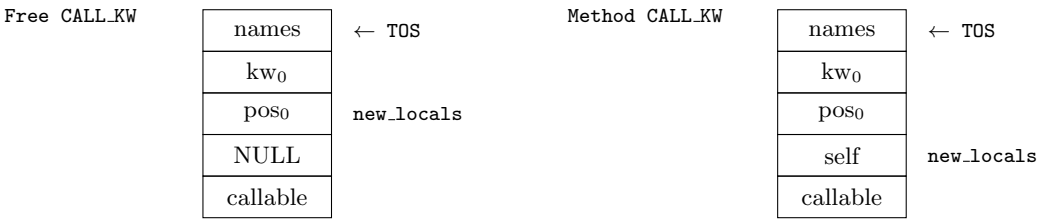

Figure 3: Illustrative RF layouts after names are still on stack (one positional, one keyword). Method form places `self` where free calls place NULL; the binder's kwargs value base is therefore `argcount` ($= n_{\text{pos}} + 1$), not n_{pos}.

6 Code-object metadata consumed by CALL

The serialized code object carries a packed metadata integer:

Bits	Field
[15 : 0]	co_argcount
[31 : 16]	co_nlocals
[47 : 32]	co_stacksize
[63 : 48]	co_kwonlyargcount
[64]	CO_VARARGS
[65]	CO_VARKEYWORDS
[81 : 66]	co_posonlyargcount

Companion heap fields include co_defaults (tuple), co_varnames (tuple), and co_kwdefaults (dict). CPython 3.14 varname order for binding is: positionals, keyword-only, then *args / **kwargs names. The binder searches only the formal prefix argcount + kwonlyargcount; vararg names are never keyword targets—a keyword spelled like args lands in **kwargs when present.

7 The shared binder

7.1 Definition

The *binder* is the nested micro-sequence under phase 14 (subs 32–55) that implements CPython argument binding for CODE_OBJECT callees. CALL, CALL_KW, and post-expand CALL_FUNCTION_EX share it. Phase 6 forces the binder whenever the callee has keyword-only parameters *or* CO_VARKEYWORDS, even for a purely positional call—because an empty **kwargs dict local must still be installed.

7.2 Pipeline

Figure 4 summarizes the binder pipeline.

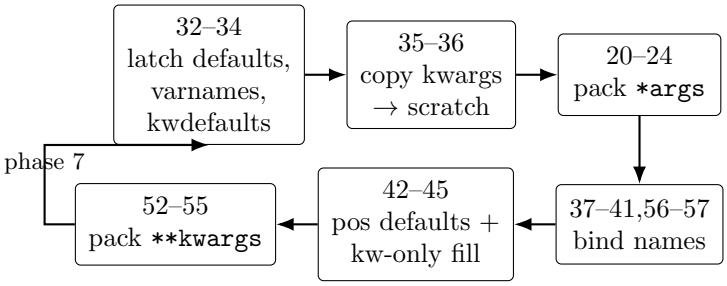

Figure 4: Shared binder micro-sequence (subs under phase 14). Calls without keywords skip the scratch copy; callees without CO_VARARGS/CO_VARKEYWORDS skip the corresponding packs.

7.3 Keyword match rules

For each caller keyword name n:

1. Scan formals $k \in [0, \texttt{total\_params})$.
2. If n matches formal k and $k <$ posonlyargcount, treat as *unexpected* (positional-only).
3. If n matches and the slot is already filled $\Rightarrow$ CALL_FILTER (duplicate), even when **kwargs exists.
4. If n matches an unfilled non-posonly formal $\Rightarrow$ write the value and mark filled.

5. If no formal matches ⇒ unexpected: record in the leftover bitmask when `CO_VARKEYWORDS`, else `CALL_FILTER`.

7.4 Variadic packing

- `*args`: excess positionals are copied into a new tuple installed at local index `total_params`.
- `**kwargs`: leftovers are inserted into a *pre-sized* dict (capacity from caller keyword count) so binding never raises `DICT_GROW`; the handle is installed after positionals, kw-only, and `*args`.

8 Worked examples

8.1 Positional defaults

```
def f(a, b=2):
    return a + b
f(1) # -> 3
```

Mode `POS`, no binder force. Phase 14 fills b from `co_defaults`, bumps `call_argcount` to `co_argcount`, then phase 7 clears only true temporaries.

8.2 Keyword reorder

```
def g(a, b):
    return 10*a + b
g(b=4, a=3) # -> 34
```

`CALL_KW`: names `('b','a')`, values below. Binder matches by `co_varnames` and writes locals $[3, 4]$.

8.3 Starred call with empty kwargs

```
def h(a=0, **k):
    return a + len(k)
h(*(), **{}) # -> 0
```

EX expands to a zero-positional call; `CO_VARKEYWORDS` forces the binder; subs 52–55 install an empty dict; defaults supply $a = 0$.

8.4 Positional-only

```
def p(a, /, b=0, **k):
    return a + 10*b + k.get("a", 0)
p(1, a=2) # a=1, k={"a": 2}
p_posonly = lambda a, /: a
p_posonly(a=1) # CALL_FILTER
```

8.5 Bound method + keywords

```
# o.m is method-form: callable, self, args...
def add(self, a, b=0):
    return a + 10*b
o.m(1, b=2) # -> 21
```

Incoming kwargs sit at `locals`[`argcount` $+ i$]. Using n_{pos} as the source base would read `self` or a positional—a latent hole closed when method+kwargs tests landed.

9 Important design details

9.1 Latched kwargs scratch bases

Keyword values are copied from the incoming stack region into a scratch region *above* real locals so `*args`/`**kwargs` packing cannot clobber them. The scratch base must be **latched in binder sub 34**. A combinational function of `call_argcount` is unsafe: the varargs pack bumps argc, which would move the scratch window and cause later binder reads to miss parked values (observed as a `TYPE` trap in `*args`+kw-only tests).

Registers:

- `call_kw_val_base_r` — source index of incoming kwargs (equals effective argc at latch time; includes `self` for methods).
- `call_kw_scratch_base_r` — destination index, at least `argcount` + n_{kwargs}, and at least past `*args`/`**kwargs` local slots when those flags are set.

9.2 Effective argc after default / variadic fill

Phase 7's UNINIT clear uses [`call_argcount`, `nlocals`). Any value the binder installs into a local—defaults, `*args`, `**kwargs`—must be reflected in `call_argcount` before phase 7, or the clear wipes it. This is the argc==0 defaults-wipe class of bugs; the same discipline applies to empty `**kwargs`.

9.3 Pre-sized `**kwargs` dict

Leftover packing sizes the dict for the caller's keyword count up front. Binding therefore stays on the pycore path without an excore `DICT_GROW` handoff. The leftover bitmask is 128 bits, matching the binder's 7-bit keyword index walk.

9.4 Fatal filters instead of exception objects

Bad call shapes raise `PY_TRAP_CALL_FILTER` and halt the simulation / prototype. That matches PyCore's current exception story (`RAISE_VARARGS` is also fatal-minimum) and keeps the CALL path free of heap-allocated `TypeError` instances.

9.5 Builtins and kwargs

`OBK_BUILTIN` and raw type objects still reject caller kwargs with `CALL_FILTER`. Keyword-capable builtins are expected to be ROM firmware `CODE_OBJECT`s (e.g. `print`) so they reuse the same binder.

10 Tradeoffs

Cycle cost vs. semantic fidelity. PyCore deliberately spends cycles on name compares and dict probes rather than approximating "positional only forever." The research claim is that a bytecode-native CPU can host real CPython call shapes; the binder is where that claim is won or lost.

Testing strategy. Differential image tests (`run_image_test.py`) execute the same source on CPython 3.14 for a golden tag/value, then in Verilator. Trap tests encode expected `CALL_FILTER` without host execution. The suite `pycore-img-call-all` is the regression gate for this FSM.

Choice	Benefit	Cost / alternative
Shared binder for CALL / CALL_KW / EX	One CPython-shaped semantics path; EX becomes expand-then-bind	Larger phase-14 sub-FSM; harder to reason about in isolation
Function ≡ code object	Simple image model; defaults on the code handle	No per-closure function objects yet; `MAKE_FUNCTION` attributes limited
Pre-sized `**kwargs`	No grow trap during bind; deterministic cycle bound for n_{kw}	Over-allocates when many keywords bind to formals; 128-kw leftover cap
Latch scratch bases	Correct under argc mutation from `*args`	Extra registers; easy to regress if a new path recomputes combinationally
Fatal `CALL_FILTER`	Small RTL; clear differential tests against host `TypeError`	Not source-compatible with `try/except TypeError` yet
Builtin kwargs via firmware	Avoids per-`BI_*` keyword tables in RTL	Builtin fast path and firmware path must stay behaviorally aligned

Table 1: Design tradeoffs in the CALL / binder implementation.

11 Limits and non-goals

- No rich exception objects or exception handlers on the CALL path.
- Leftover-keyword bitmask width 128 (binder index width).
- Non-string / contaminated EX kwargs keys trap during bind.
- Closures / cell variables are outside the interim function model.
- Full LEGB for nested scopes remains a separate initiative.

12 Reference map

Artifact	Role
`pycore/rtl/pycore_call_fsm.svh`	Phase/binder implementation
`pycore/rtl/pycore_core.sv`	`S_CALL` state, mode/phase locals, scratch latch wires
`pycore/rtl/pycore_defs.svh`	Metadata bitfields, trap codes, opcodes
`pycore/tools/encoding.py`	Metadata pack/unpack
`planning/co_varkeywords_call_parity.md`	Recent parity delta + test matrix
`pycore/docs/bytecode_support.md`	Opcode support matrix
`Makefile` `pycore-img-call-all`	Hardware regression aggregate

13 Conclusion

The CALL FSM is the semantic gate between CPython's calling convention and PyCore's frame/locals machine. Its important idea is not a clever opcode encoding but a *shared binder*

that makes `CALL`, `CALL_KW`, and `CALL_FUNCTION_EX` converge on one CPython-faithful binding procedure, with explicit handling for defaults, positional-only parameters, and variadic locals. With that path complete for user code objects, subsequent paper systems—ROM firmware builtins, richer objects, and language-level features that assume ordinary calls—can build on a stable foundation.